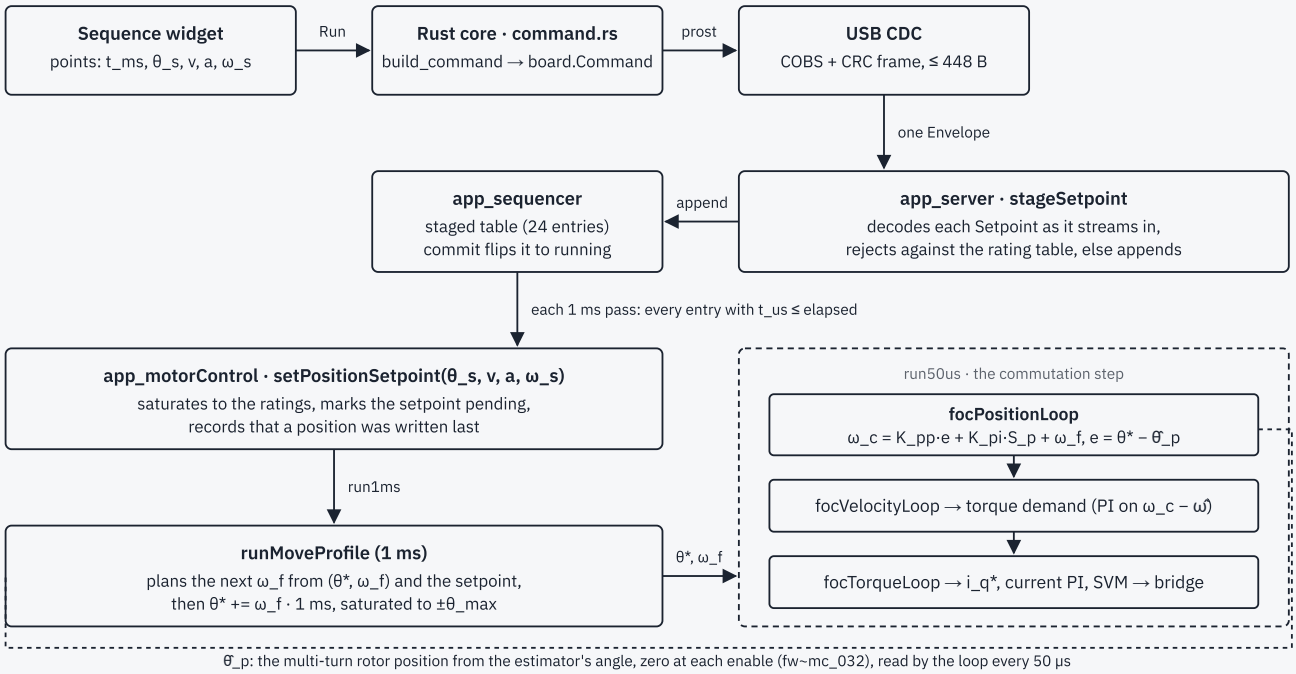
PCS_BLDC · MC_FOC · 2026-11-11

Position Trajectory Path

How a handful of points typed into the desktop app become a motion profile on the STM32, and how that profile is split between the 1 ms task and the 50 µs commutation step. Ends with the bench capture that shows the whole chain working.

The path in one picture

The host never sends a trajectory. It sends the *sparse points* with per-point limits, and the board turns them into a dense profile itself, one millisecond at a time. Every box below is a real module; the arrow labels are the real rates and sizes.



One Command carries the whole sequence; the board plays it on its own clock and does all of the profiling. The dashed box runs at the PWM rate; everything above it at 1 ms or slower.

1. What the operator types

Both widgets key their fields off one per-kind table (`SETPOINT_KINDS` in `commandwidget.js`). Selecting the position method relabels the setpoint to rad, bounds it at the board's reported maximum position, and reveals two extra limit fields. The bounds are not guessed: the app sends `GetMotorInfo` on connect and the board answers with its five ratings.

FIELD	UNIT	BOUND (FROM MOTORINFO)	EMPTY OR ZERO MEANS
Position θ_s	`rad`	±max_position_rad (62.83, ten turns)	the enable point
Velocity limit v	`rad/s`	max_velocity_radps (18.0)	**max**: cruise at the rating
Acceleration limit a	`rad/s²`	none	**step**: ω_f jumps to ±v
Velocity at the point ω_s	`rad/s`	the velocity limit (the rating when v is zero)	**stop**: rest on the point

The Command widget sends one such setpoint at time zero. The Sequence widget holds a list of them as points on a chart, each with its time from the sequence's start, and draws the target the board will actually follow. That preview is not a sketch: `positionTrajectory()` in `sequencewidget.js` replays the firmware's own 1 ms planner step for step, so what the chart shows is what θ* will do on the board, to the millisecond.

Run packs the points, in time order, into one `CommandSpec` for the Rust core. A position point goes out as `position_rad` with `rate_limit` carrying v, plus `accel_limit_radps2` and `velocity_at_point_radps`.

2. The wire

The protocol is one `Envelope` per frame. A board command rides in its `board_request` arm; a `Command` is a mode plus a repeated `Setpoint`. The setpoint's target is a `oneof`, one arm per kind, and the limits sit beside it as plain fields.

```
message Setpoint {
  uint32 t_us = 1;                    // from the sequence's start
  oneof target {
    float velocity_radps = 2;
    float torque_nm      = 3;
    Stop  stop           = 5;         // MODE_OFF at t_us
    float position_rad   = 6;
  }
  float rate_limit              = 4;  // rad/s², Nm/s — or a position's velocity limit in rad/s
  float accel_limit_radps2      = 7;  // position only; zero steps
  float velocity_at_point_radps = 8;  // position only; zero stops
}
message Command { Mode mode = 1; repeated Setpoint setpoints = 2; }   // MODE_FOC_POSITION = 5
```

Reusing `rate_limit` keeps the message uniform: it is always "per second in the kind's unit", and for a position that unit is rad, so the rate is a velocity. One position setpoint on the wire, 2π rad at 200 ms with v = 10, a = 50, ω_s = 0:

```
Request.run (2) 12 17 · Command.mode 08 05 · Command.setpoints (2) 12 13
```

```
t_us 08 c0 9a 0c (varint 200000) · rate_limit 25 00 00 20 41 (10.0) · position_rad 35 da 0f c9 40 (6.2832) · accel_limit 3d 00 00 48 42 (50.0)
```

25 bytes; ω_s = 0 is left off the wire, as proto3 does with defaults. A frame carries up to 448 B of payload, so about sixteen position setpoints fit in one Command.

Mode is the method the board runs the sequence under. The board accepts a method only while the bridge is disabled or already running that method, so a sequence can be re-sent mid-run but cannot switch methods under a live bridge.

3. Into the sequencer

On the board the setpoint list is never held as a message. nanopb decodes it through a callback, one `Setpoint` at a time, and `app_server_config`'s `stageSetpoint` checks each against the rejection table and appends it straight into the sequencer's *staged* table. The first fault found rides to the verdict; the rest still decode so the stream stays in sync.

A POSITION SETPOINT IS REJECTED WHEN	VERDICT CAUSE
|θ_s| exceeds the maximum position	`setpoint exceeds the motor rating`
v exceeds the maximum velocity	`velocity limit exceeds the motor rating`
|ω_s| exceeds v (or the maximum velocity when v is zero)	`velocity at the point exceeds the velocity limit`
any value not finite, or v or a negative	`setpoint not finite / rate limit negative`
its time precedes the point before it, or the list passes 24 entries	`setpoint times not in order / sequence exceeds the player's capacity`

The sequencer owns two halves of one array: the *running* table the 1 ms pass reads and the *staged* table the server fills. `begin` empties the staged half, `append` adds entries, and when the whole Request has decoded clean the handler calls `commit` inside a critical section together with the mode request, so the sequence starts under its mode. The commit only raises a flag; the halves flip on the next 1 ms pass, which is pass 0 of the new sequence. A second Command inside that millisecond is rejected as busy and the host resends.

The 1 ms pass

Each pass, while a sequence is running, the player walks the running table from the next

unwritten entry and writes every entry whose `t_us` is at or under the elapsed time, in list order. A position entry is written as one call:

```
case APP_SEQUENCER_KIND_POSITION:
    app_motorControl_setPositionSetpoint(motor, value, rateLimit, accelLimit, velocityAtPoint);
```

That is the whole hand-over. The sequencer knows nothing about profiles; it delivers each sparse point at its time and forgets it. A latched fault drops the unwritten rest. When the last entry is written the sequence is over, but the motor control keeps whatever the last point asked for: a stopped point holds its position, a point with ω_s keeps rolling at ω_s until something else is written.

4. Where the trajectory is made: two rates inside app_motorControl

The setter does almost nothing. It saturates the four numbers to the ratings (v of zero becomes the maximum velocity, |ω_s| is capped at v), stores them, sets a *pending* flag, and records that a position was the most recent write. Everything that follows runs on the two periodic tasks.

run1ms · the move profile (fw~mc_031)

The 1 ms task owns the position target θ* and the profile velocity ω_f. Each pass it picks ω_f by what was written last, then advances the target by one millisecond of that velocity:

WRITTEN LAST	Ω_F THIS PASS
A position setpoint	The minimum-time profile toward θ_s (below)
A speed target, while the position method is active	The speed target in effect: a jog, θ* just keeps moving at that speed
Nothing since the enable	Zero: θ* and the rotor hold at the enable point

The profile is not precomputed. Each pass replans from the current (θ*, ω_f), so a point rewritten mid-move simply changes where the next step heads. The planner is five lines:

```
d = θ_s − θ*,  s = sign(d),  r = |d|
allow = −a·T/2 + sqrt((a·T/2)² + ω_s² − a·T·ω_s + 2·a·r)   // the speed from which braking at a reaches ω_s
ω_f ← ω_f slewed toward s · min(v, allow) at a per second      // a = 0: ω_f = s·v outright
if (ω_f·s ≥ 0 and r ≤ |ω_f|·T): arrived — ω_f = s·ω_s, and θ* = θ_s if ω_s = 0
θ* ← θ* + ω_f·T, saturated to ±θ_max (ω_f zeroed when it would push past a bound)
```

The square root is the braking curve, written for the discrete step so that following it lands exactly on the point. While far away it is larger than v, so the slew runs ω_f up to the cruise; as the point nears it drops below v and the slew follows it down to ω_s. After arrival the

velocity holds at ω_s in the direction of approach, which is what lets the next point take over at speed. The desktop preview is this same block in JavaScript.

run50us · the position loop (fw~mc_029)

The commutation step reads θ* and ω_f and runs the cascade. The rotor position θ̂_p is a multi-turn count the same step maintains from the estimator's angle: turns counted at the ±π seam, plus the angle less its value at the enable, so there is no accumulation error at any travel.

```
ω_c = K_pp·e + K_pi·S_p + ω_f,    e = θ* − θ̂_p,    S_p ← S_p + T·e
```

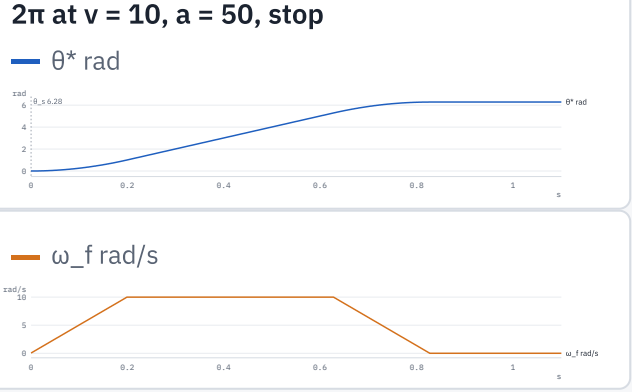
ω_f is the feed-forward: on a well-tracked move the PI terms are near zero and the velocity loop is simply told the profile's speed. The sum is clamped to ±ω_max; a clamped step leaves S_p where it was rather than back-calculating it, because with ω_f already at ω_max back-calculation would wind the integral against the proportional term and leave the loop limp when the profile stops (this is what the first bench capture showed). ω_c then goes into the velocity method's step unchanged: a PI on ω_c − ω̂ sets the torque demand, the torque loop turns it into i_q* and the current regulators drive the bridge. Gains on the board: K_pp = 30 1/s, K_pi = 90 1/s².

> **So the split is:** the host sends points; the server validates and stages them; the sequencer delivers each point at its time; the 1 ms profile turns the latest point into a dense (θ*, ω_f) stream; the 50 µs loop tracks that stream through the existing velocity and torque loops. No layer above run1ms ever sees a trajectory.

5. Basic trajectories

These are the planner above run in this page for a few setpoints, each from rest at θ* = 0 with the board's ratings (ω_max 18 rad/s). The same numbers are the acceptance bullets of fw~mc_031.

2π at v = 10, a = 50, stop

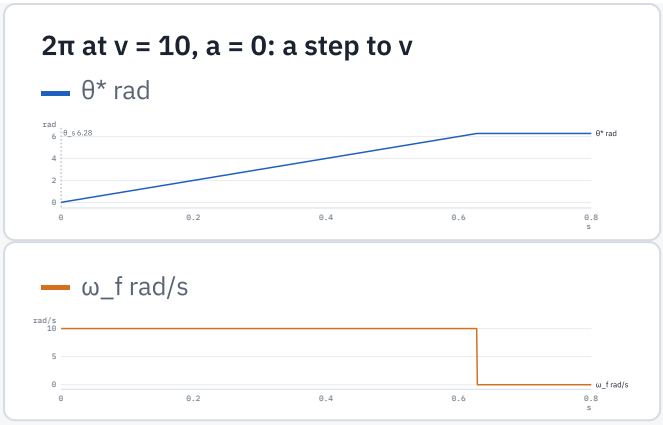


accelerates 200 ms, cruises, decelerates onto the point at 828 ms

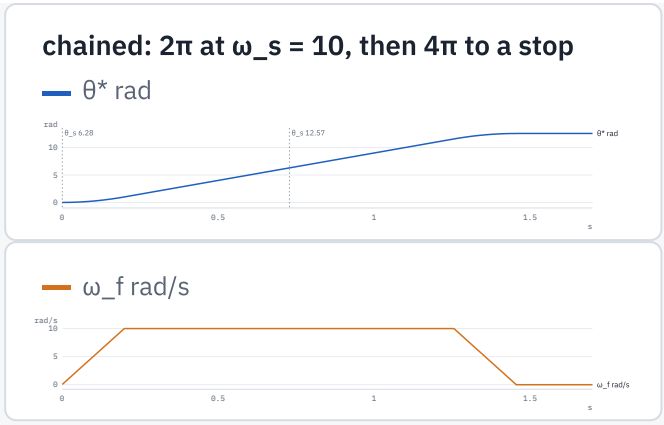
0.5 rad at the same limits: a triangle



never reaches v; peaks at 5 rad/s at 100 ms, rests at 200 ms



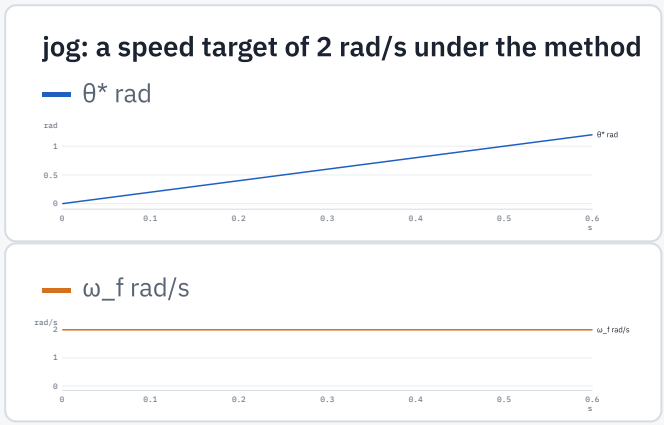
ω_f jumps to 10 on the first pass and snaps to rest at 628 ms



passes 2π at 10 rad/s at 728 ms and carries straight on to 4π



decelerates through zero at 50 rad/s² and comes back to rest on 0

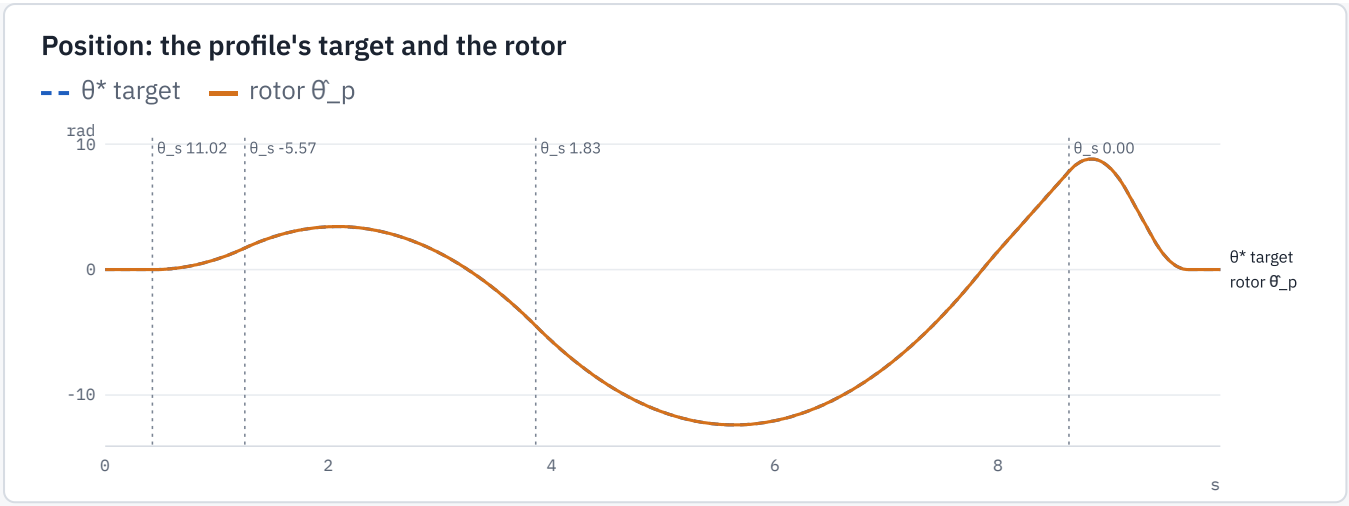


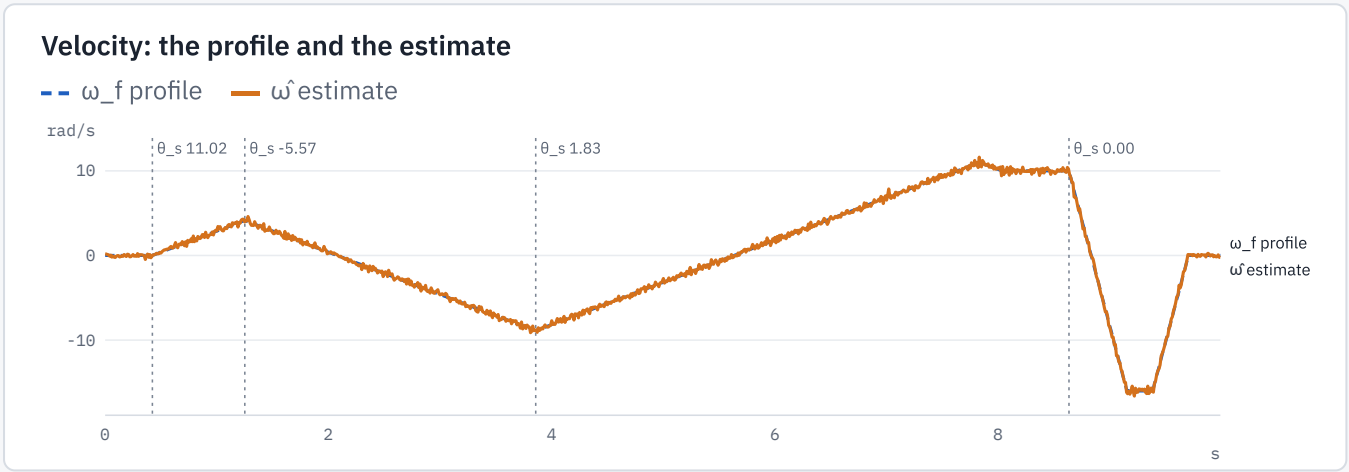
ω_f is the speed target in effect; θ* advances 2 mrad every pass until something else is written

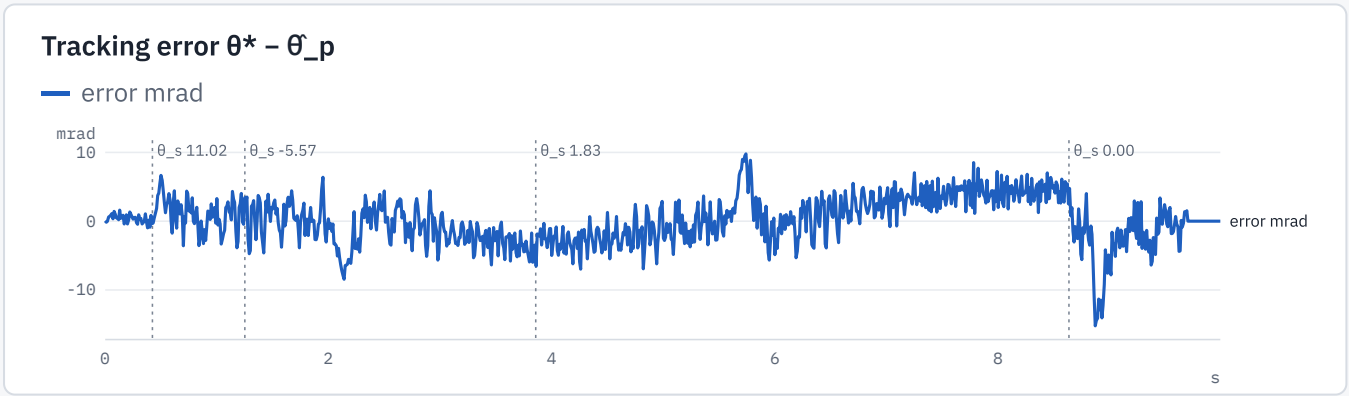
6. The bench capture, end to end

Capture `pos_trajectory4.csv`, 2026-11-11: a four-point sequence sent from the Sequence widget and traced on the board at 1 ms. The third point carries a velocity at the point of 10 rad/s, so the profile passes through it without stopping. Points, as written by the sequencer:

WRITTEN AT	Θ_S	V	A	Ω_S	WHAT HAPPENED
0.423 s	11.023 rad	16	5	0	rewritten before it was reached
1.251 s	-5.567 rad	16	5	0	rewritten before it was reached; the profile had just reversed
3.857 s	1.835 rad	16	5	10	ω_s = 10: reached at 8.03 s at 10.00 rad/s, ω_f held at 10 for the next 600 ms
8.634 s	0.000 rad	16	50	0	chained from 10 rad/s through a −16 rad/s cruise, at rest on 0 at 9.69 s







The error panel is the whole story of the loop: a few mrad through 16 rad/s cruises, 16 mrad at the start of the 50 rad/s² reversal, and 4.5 mrad as θ* crosses the third point at 10 rad/s. The rotor is on the profile the firmware generated from four numbers per point, and the profile matches what the widget drew before Run was pressed.

The four bench runs that got here

RUN	WHAT IT SHOWED	CHANGE
`pos_trajectory`	10 rad/s moves on the profile within mrad; at the 20.9 rad/s rating the voltage ceiling held the rotor 0.5 rad/s short and the clamp's back-calculation wound the integral, so stops crept for up to a second	rating to 18 rad/s; integral held through the clamp

`pos_trajectory2`	creeps gone; at exactly ω_max the ramp's lag is frozen through the cruise (18 to 65 mrad) since the feed-forward fills the clamp	none; use v under the rating for headroom
`pos_trajectory3`	v = 16 rad/s: 5 to 7 mrad at cruise, 3 to 8 mrad at every arrival, within 0.01 rad at the stop	none
`pos_trajectory4`	a point passed at 10 rad/s and chained into a stop at 0: the ω_s clause of fw~mc_031 on the bench	none